Linear models induce uniform trajectories

We suppose that linear models can develop fuller, more uniform rotations, and that nonlinearity pushes the distribution of solutions into less organized, more independent trajectories.

We test this in two ways:

1. initialize a new linear RNN on phoneme output given the final time step delay period hidden state from a nonlinear parent model
2. train a linear RNN on end-to-end phoneme recall

Let time t_f be the final output time step for the source model. We then take $\tilde{\mathbf{h}}_{t_f}$ from the parent model and let $\mathbf{h}_0 = \tilde{\mathbf{h}}_{t_f}$. Our linear RNN $\mathbf{h}_0$-initialized RNN has the formulation:

$$\mathbf{h}_t = \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}.$$

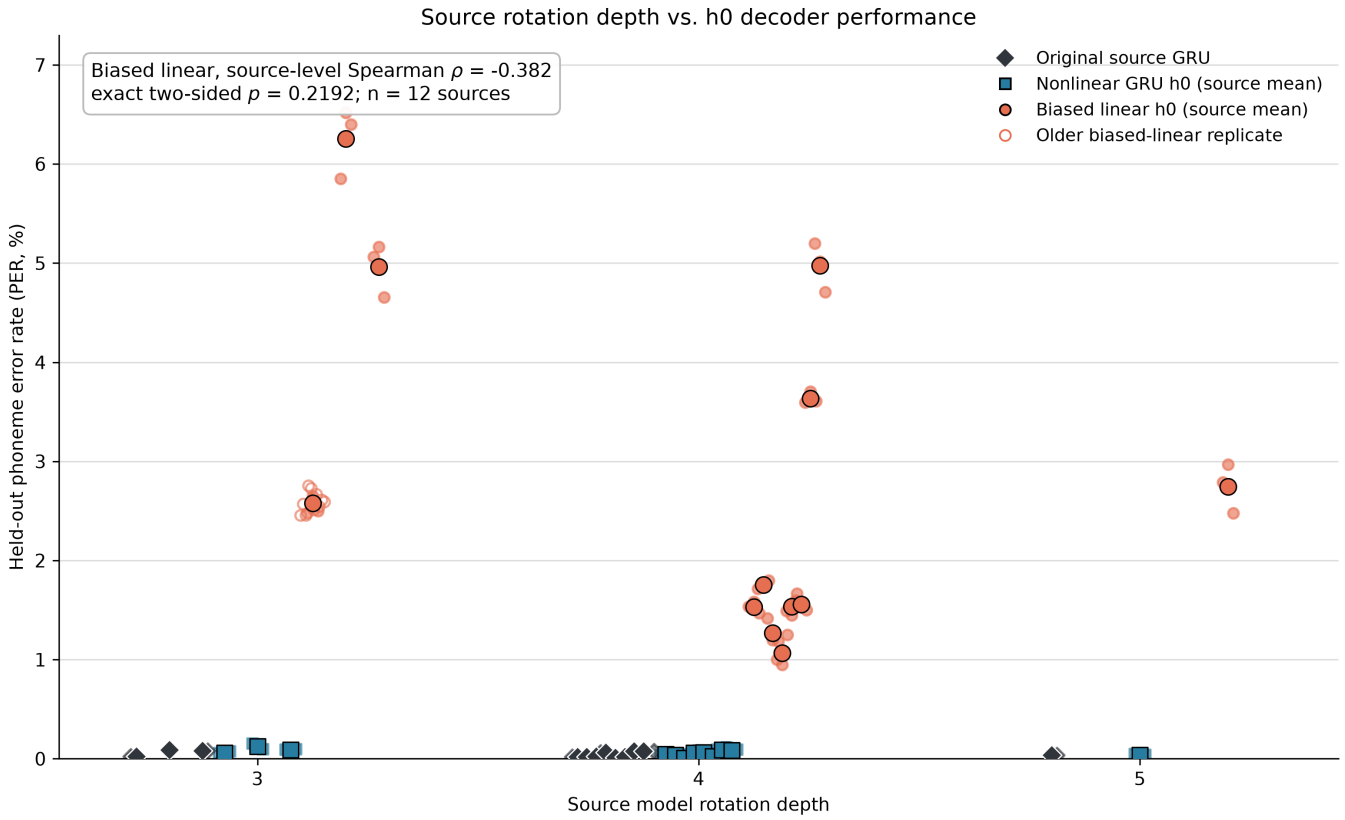
We compare this to the formulation which trains a GRU architecture with $\mathbf{h}_0$ initialized. The motivation is that we can see whether output architecture alone accounts for the distinction in geometry, or whether hidden state organization plays a role. From previous analysis, we know that both are in play.

As source models, we use $l \leq 7$, delay period $m = 0, 2, 4, 6$, hidden state size $d = 384$ runs trained on 3 seeds. For each source model, we use 3 seeds to initialize dataset splits and training weights for their $\mathbf{h}_0$-initialized derivatives. We find that for instances where the source model and nonlinear $\mathbf{h}_0$-initialized derivative models attained partial rotation depth, the linear $\mathbf{h}_0$-initialized model generally developed fully rotating geometries (depth=6). However, for longer delay periods $l = 4, 6$, some models where the source rotation depth is on the lower end $(3, 4)$ yielded linear derivative model rotation depths of < 6. Furthermore, the PER for linear derivative models were consistently above their nonlinear counterparts, with a dramatic increase in PER for longer delay period time steps. As the derivative models contain no delay period, such evidence indicates that there is a spectra of how favorable $\mathbf{h}_0$ is for more organized, uniform rotation outputs. However, this relationship between source model rotation depth and linear derivative PER is not statistically significant.

Delay	Source seed	Source depth	Source PER (%)	Linear h0 depth	Linear h0 PER (%)	Nonlinear h0 depth	Nonlinear h0 PER (%)
0	0	4	0.0198	6.000	1.5301	4.333	0.0446
0	1	4	0.0205	6.000	1.7543	5.000	0.0358
0	2	4	0.0237	6.000	1.2665	4.000	0.0047
2	0	4	0.0645	6.000	1.0667	5.333	0.0575
2	1	4	0.0121	6.000	1.5351	6.000	0.0649
2	2	4	0.0192	6.000	1.5592	5.333	0.0261
4	0	3	0.0259	5.667	2.5429	4.000	0.0592
4	1	4	0.0750	6.000	3.6361	4.667	0.0909
4	2	5	0.0372	6.000	2.7441	5.667	0.0381

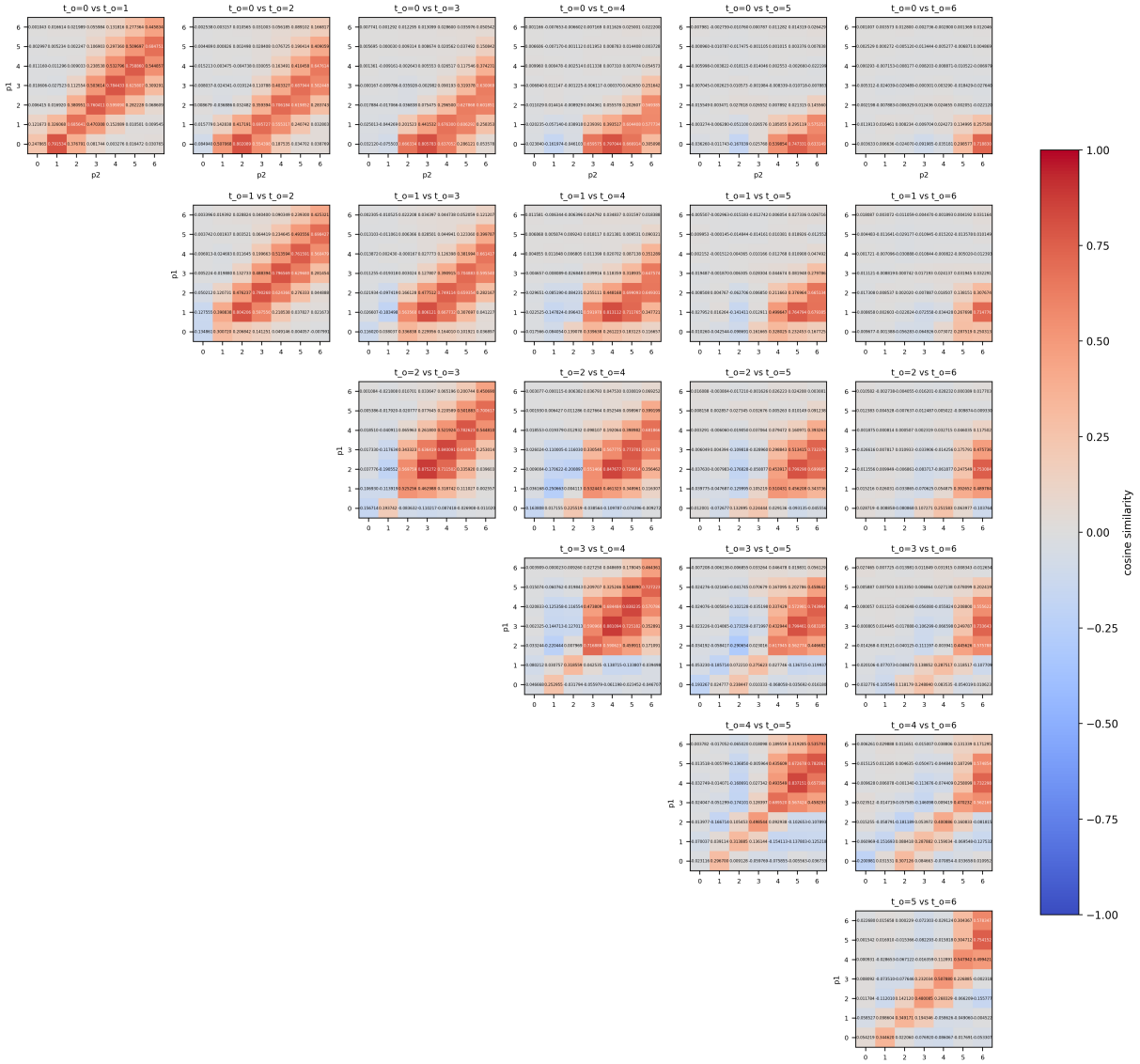
Delay	Source seed	Source depth	Source PER (%)	Linear h0 depth	Linear h0 PER (%)	Nonlinear h0 depth	Nonlinear h0 PER (%)
6	1	3	0.0885	5.000	6.2553	4.000	0.1261
6	2	3	0.0818	5.333	4.9611	4.000	0.0907
6	3	4	0.0774	5.000	4.9738	4.000	0.0869

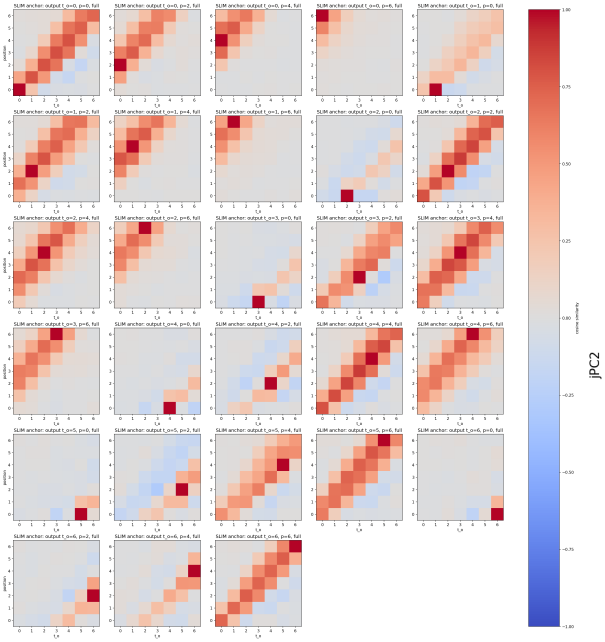
PER and rotation depths averaged across 3 seeds.

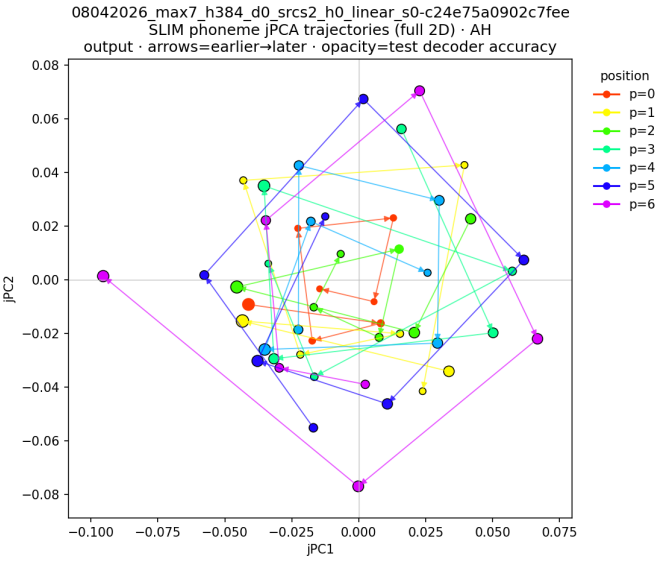


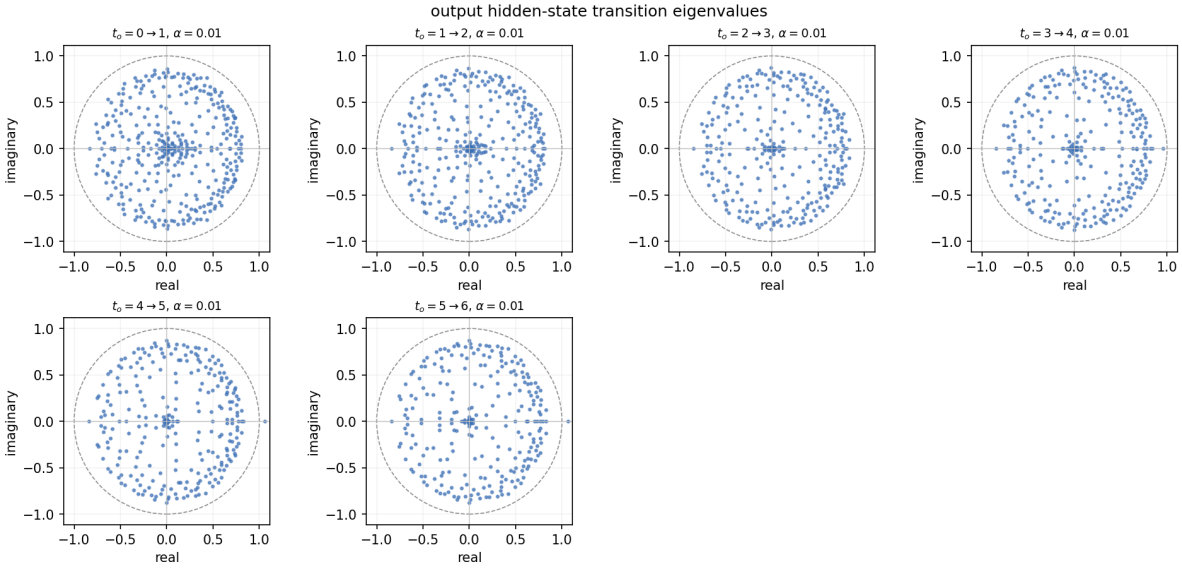
Source model rotation depth vs. test PER for source, nonlinear derivative, and linear derivative models.

Examples of linear (top 4 images) versus nonlinear (bottom 4 images) plots, sampled from a source model with partial rotations with $m = 0$.
linear:

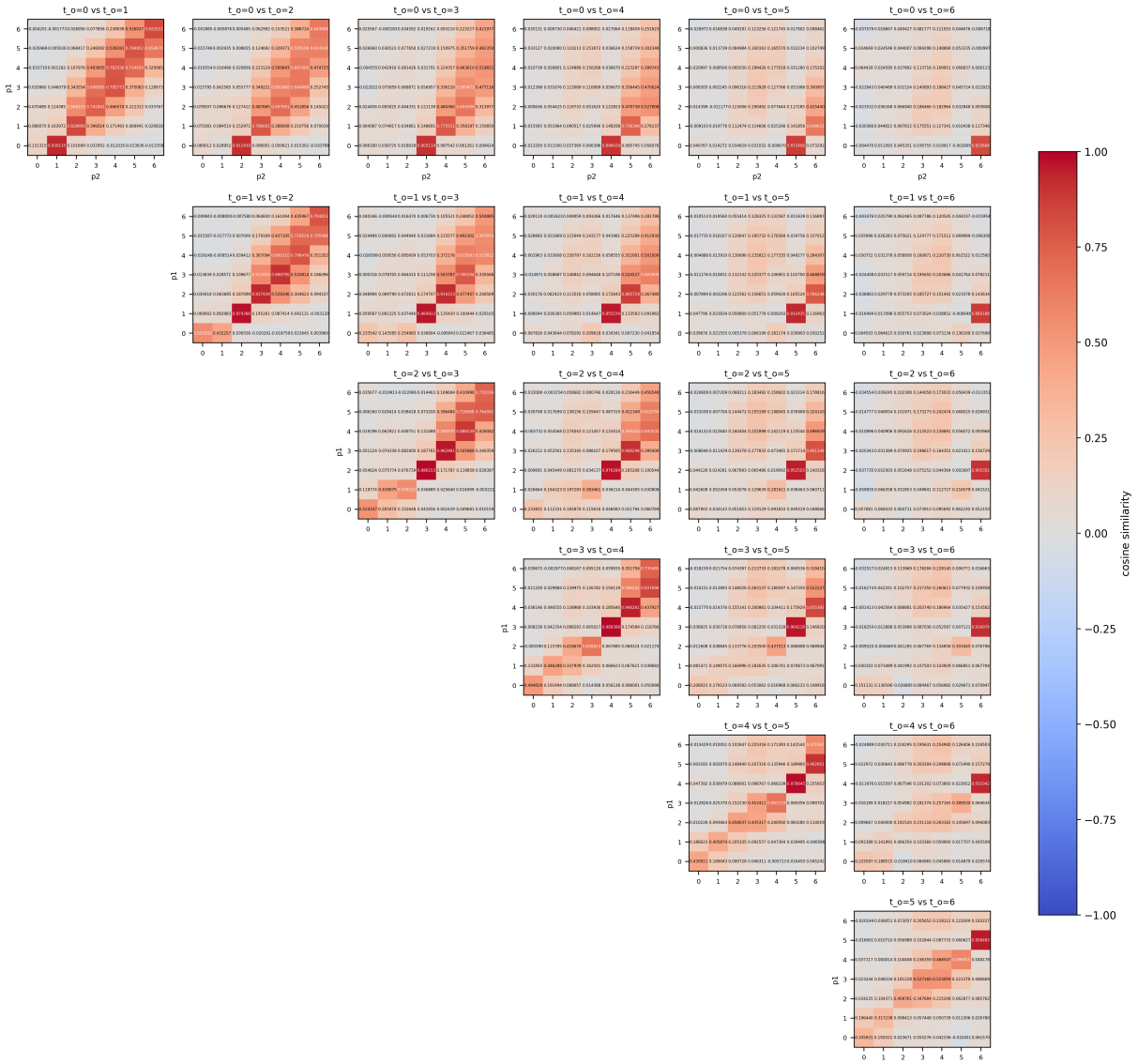

SLIM time-pair position similarity — output — natural cosine
μ=0.12643154702144074, σ=0.24294168923790002, cutoff=none
t_o=0 vs t_o=1
t_o=0 vs t_o=2
t_o=0 vs t_o=3
t_o=0 vs t_o=4
t_o=0 vs t_o=5
t_o=0 vs t_o=6
t_o=1 vs t_o=2
t_o=1 vs t_o=3
t_o=1 vs t_o=4
t_o=1 vs t_o=5
t_o=1 vs t_o=6
t_o=2 vs t_o=3
t_o=2 vs t_o=4
t_o=2 vs t_o=5
t_o=2 vs t_o=6
t_o=3 vs t_o=4
t_o=3 vs t_o=5
t_o=3 vs t_o=6
t_o=4 vs t_o=5
t_o=4 vs t_o=6
t_o=5 vs t_o=6
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

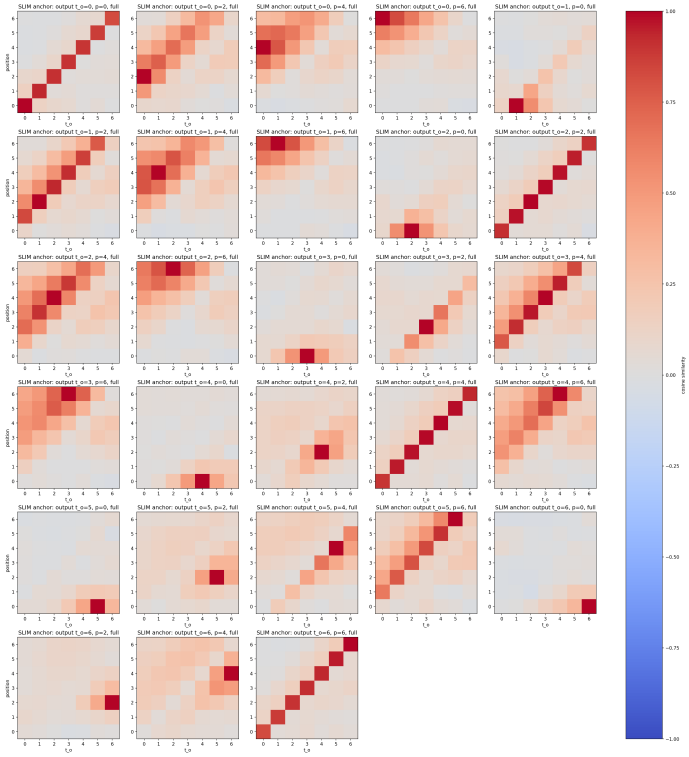


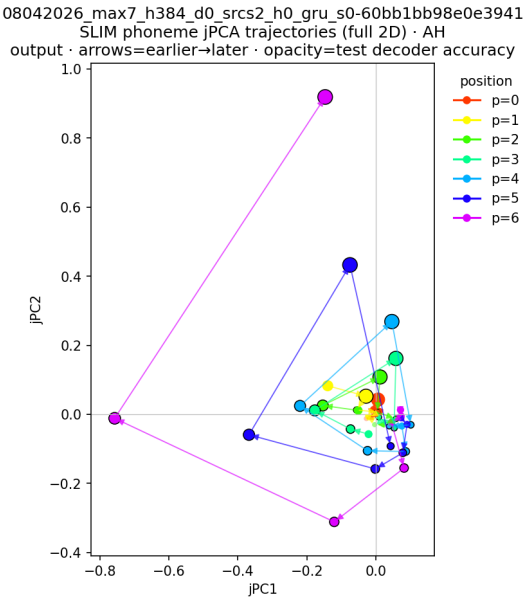

08042026_max7_h384_d0_srcs2_h0_linear_s0-c24e75a0902c7fee
SLIM phoneme jPCA trajectories (full 2D) · AH
output · arrows=earlier→later · opacity=test decoder accuracy
position
p=0
p=1
p=2
p=3
p=4
p=5
p=6
jPC1
jPC2

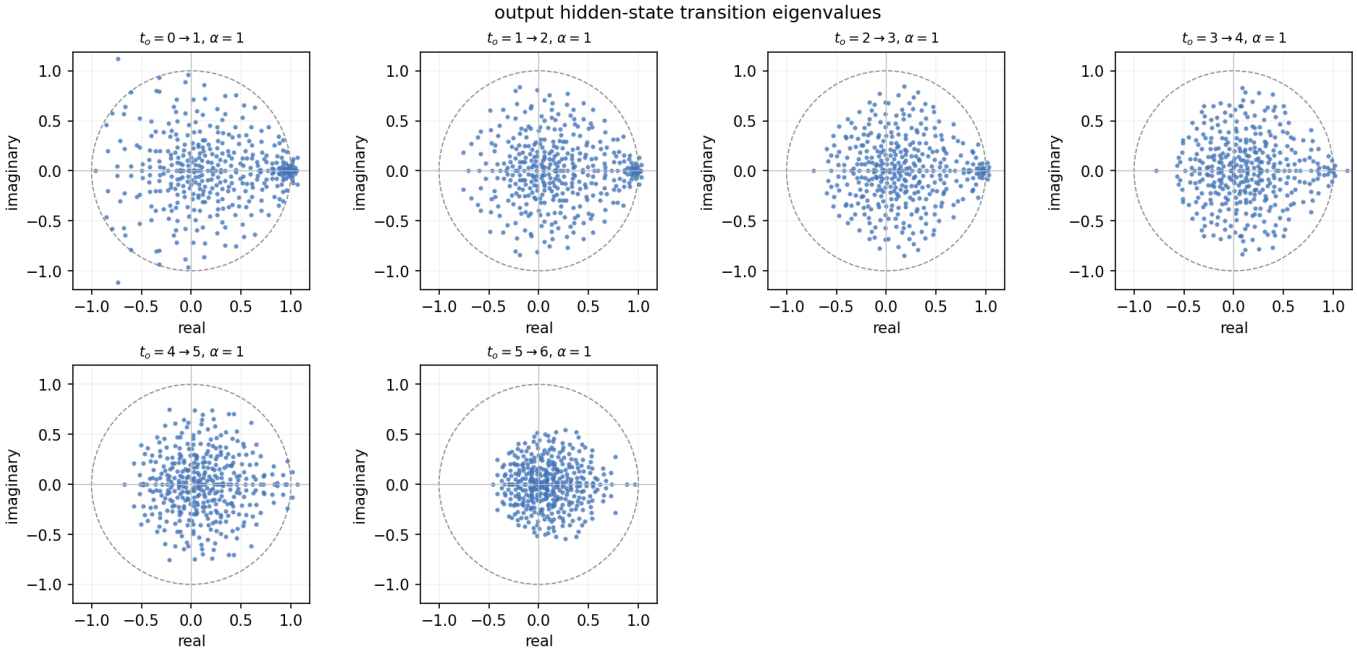

output hidden-state transition eigenvalues
$t_o = 0 \to 1,\ \alpha = 0.01$
$t_o = 1 \to 2,\ \alpha = 0.01$
$t_o = 2 \to 3,\ \alpha = 0.01$
$t_o = 3 \to 4,\ \alpha = 0.01$
$t_o = 4 \to 5,\ \alpha = 0.01$
$t_o = 5 \to 6,\ \alpha = 0.01$
real
imaginary

nonlinear:


SLIM time-pair position similarity — output — natural cosine
μ=0.16350564856714581, σ=0.21925487144287459, cutoff=none
t_o=0 vs t_o=1
t_o=0 vs t_o=2
t_o=0 vs t_o=3
t_o=0 vs t_o=4
t_o=0 vs t_o=5
t_o=0 vs t_o=6
t_o=1 vs t_o=2
t_o=1 vs t_o=3
t_o=1 vs t_o=4
t_o=1 vs t_o=5
t_o=1 vs t_o=6
t_o=2 vs t_o=3
t_o=2 vs t_o=4
t_o=2 vs t_o=5
t_o=2 vs t_o=6
t_o=3 vs t_o=4
t_o=3 vs t_o=5
t_o=3 vs t_o=6
t_o=4 vs t_o=5
t_o=4 vs t_o=6
t_o=5 vs t_o=6
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00







For end-to-end linear RNNs, we use the formulation

$$\mathbf{h}_t = \mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b},$$

with the same input and output formulation for a typical word recall task. Note that this is linear to both $\mathbf{x}$ and $\mathbf{h}$, while a bilinear formulation could look different and be able to condition $\mathbf{W}_h$ on $\mathbf{x}$ and vice versa. We use the $d = 384$, $m = 0, 2, 4, 6$ and $l \leq 7, l = 6, l = 7$ configurations, each with 3 seeds.

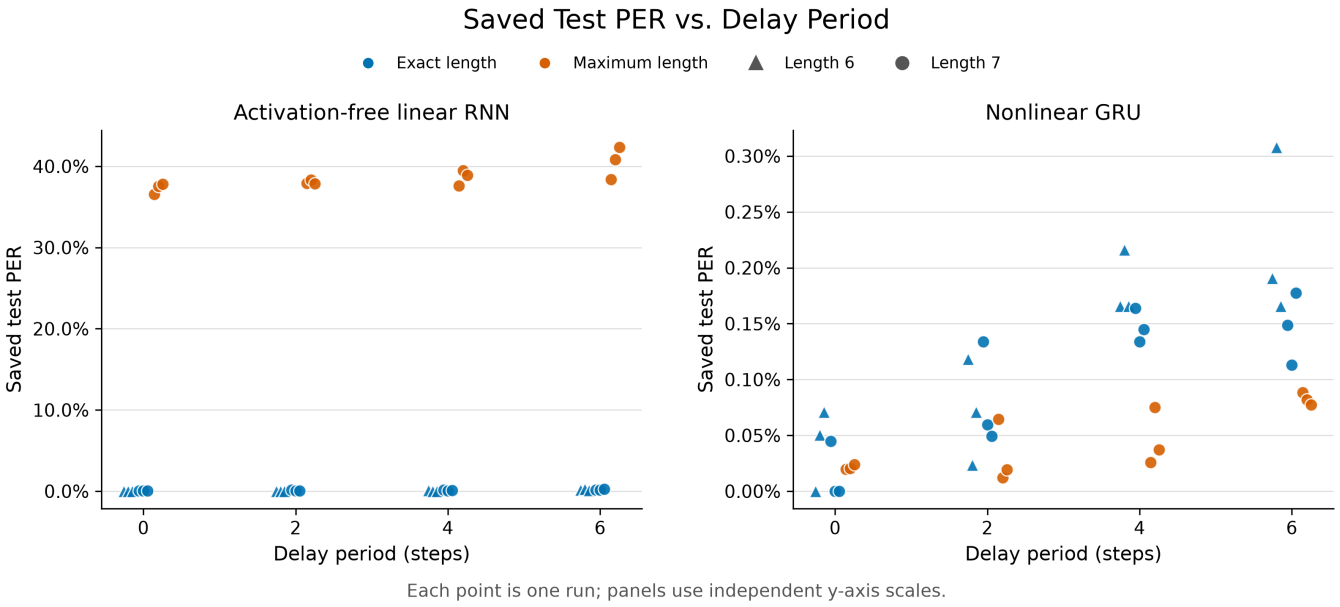
Immediately apparent is that for $l \leq 7$ training phoneme sequences, the linear RNN is unable to easily learn, with PERs around 0.4. As $\mathbf{x}$ cannot condition how $\mathbf{h}$ is processed, linear RNNs rely on one formulation for how to transform $\mathbf{h}$ between time steps for all stages—input, delay, and output. During input and output, the hidden state must rotate to accept phonemes from the input space and release phonemes into the output space, respectively. However, multiple sequence lengths cannot be handled by a

singular rotating matrix, as the number of time steps between input and output varies—what angle would we rotate by? Indeed, we observe that the PER is stratified by sequence lengths for $l \leq 7$:

Test sequence length	Delay 0	Delay 2	Delay 4	Delay 6
1	2.000	2.000	2.000	1.667
2	1.212	1.112	1.272	1.193
3	0.793	0.823	0.851	0.851
4	0.712	0.717	0.712	0.716
5	0.712	0.714	0.689	0.674
6	0.363	0.375	0.381	0.371
7	0.101	0.106	0.131	0.192

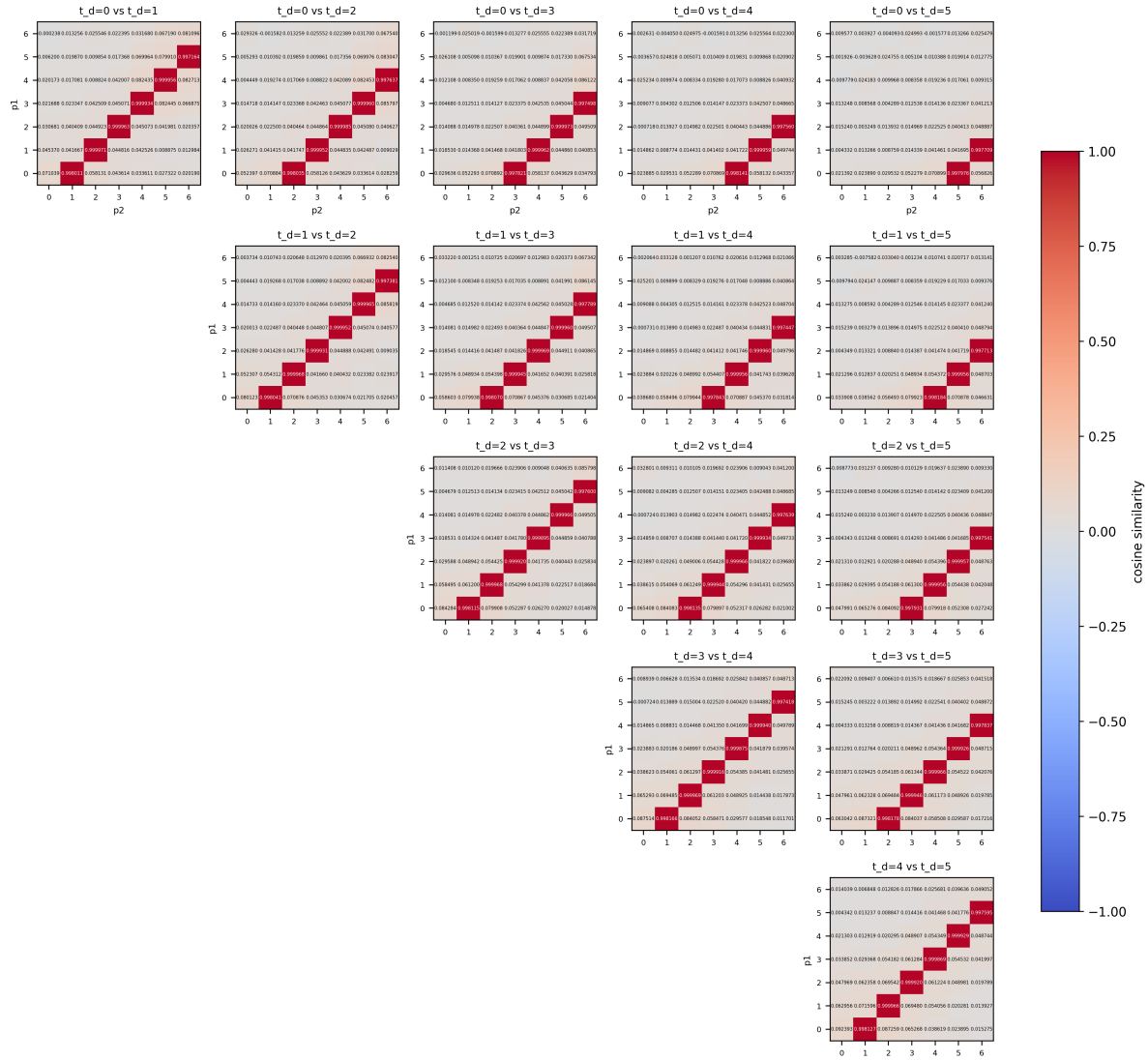
PER for various delay period lengths and test set sequence lengths for the $l \leq 7$ models.

As such, we were motivated to train $l = 6$ and $l = 7$ exact length models, with $l = 6$ as a verification that shorter sequences aren't just inherently more difficult for linear models to learn, but that the linear models can not simultaneously handle multiple sequence lengths. Appropriately, the exact length equivalents $l = 6, l = 7$ acheive much lower PERs. They also develop consistently full rotation depths (rows 599-622 in final table). As expected, the geometric solution during the delay period is to also rotate continuously rotate the phonemes in uniform slots.

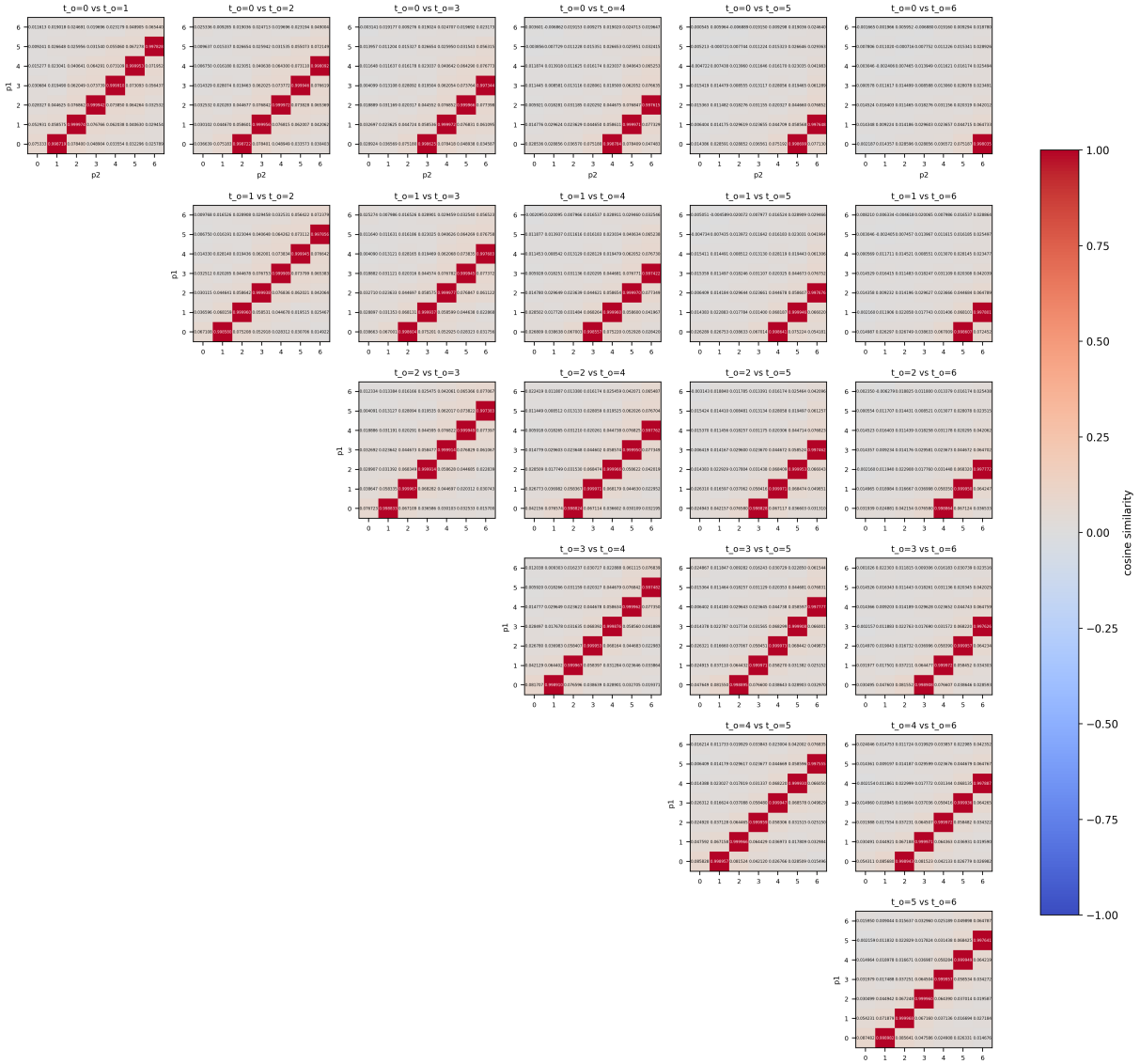


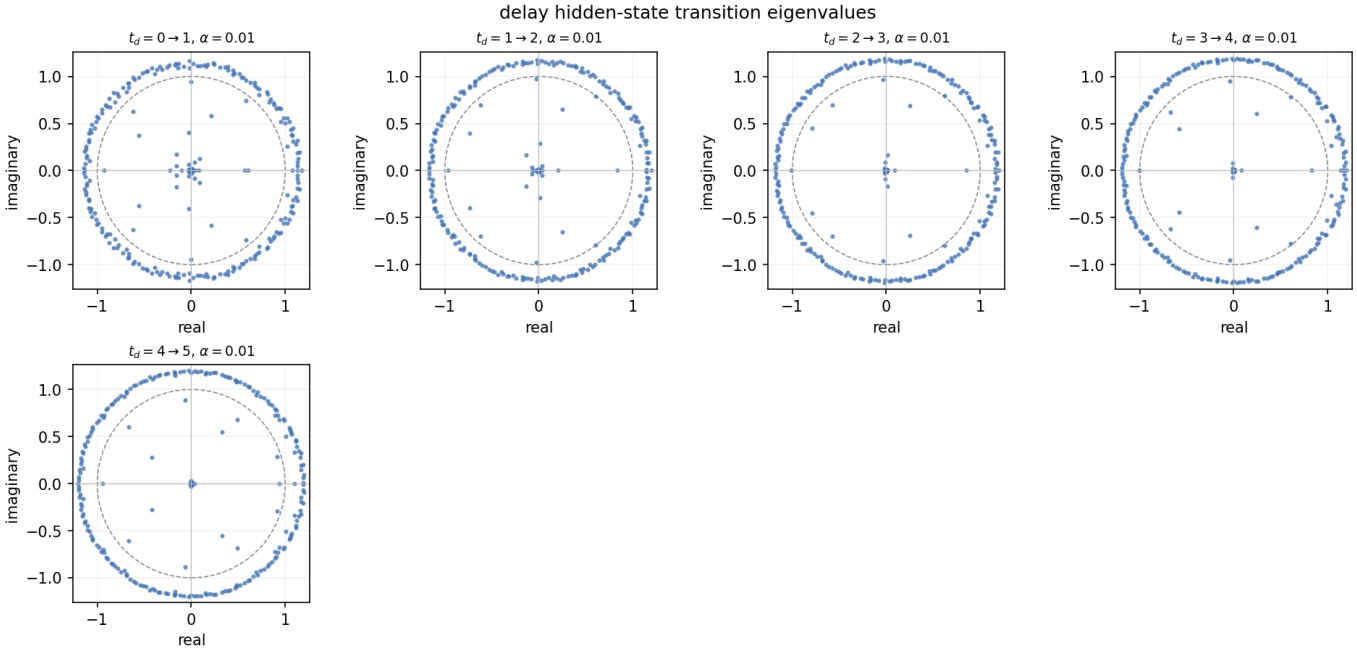
PERs for the linear models and their nonlinear equivalents, for exact length vs. maximum length schemes, by delay period length.

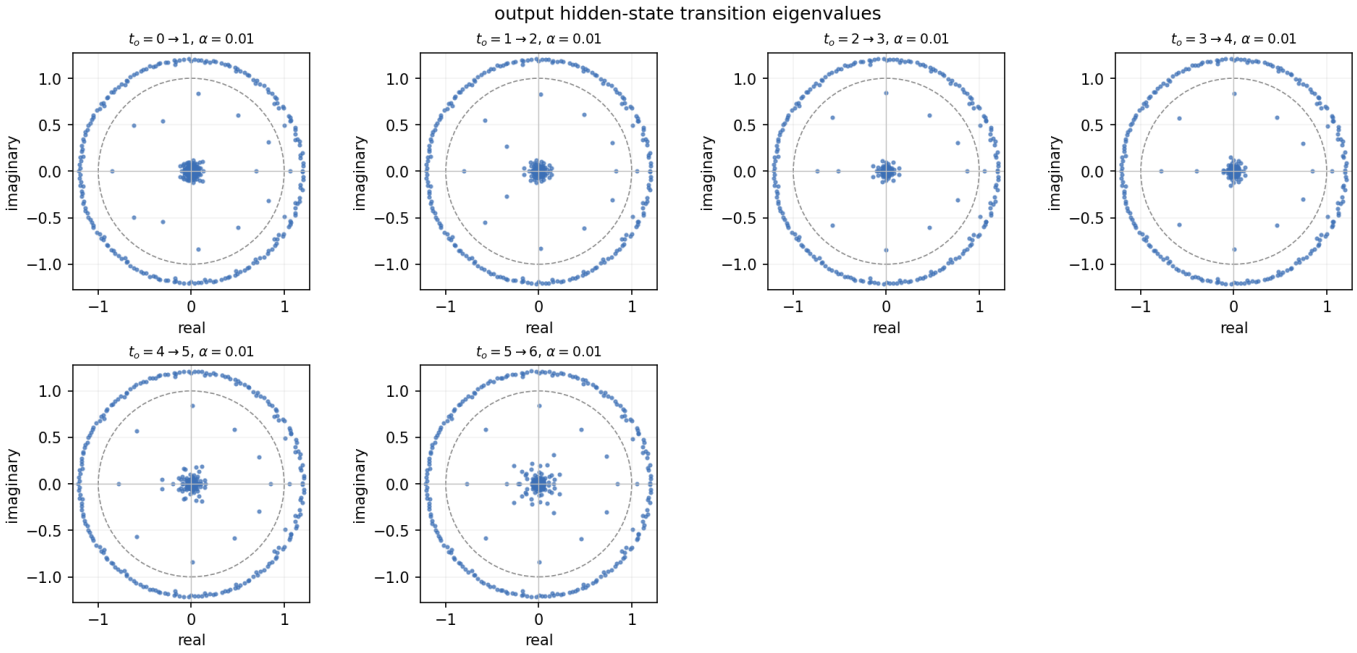
For an $m = 6, l = 7$ run, the cosine similarity between position representations for delay period (top) and output (bottom) time steps.

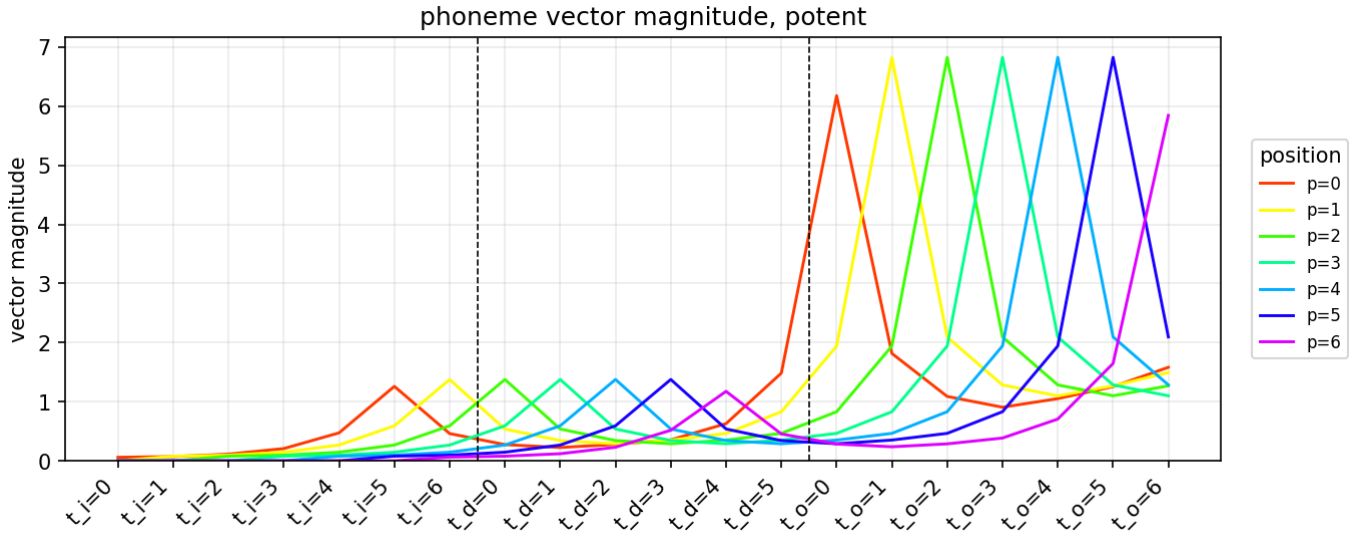

SLIM time-pair position similarity — delay — natural cosine
μ=0.12337262614218475, σ=0.28482855223238585, cutoff=none
t_d=0 vs t_d=1
t_d=0 vs t_d=2
t_d=0 vs t_d=3
t_d=0 vs t_d=4
t_d=0 vs t_d=5
t_d=1 vs t_d=2
t_d=1 vs t_d=3
t_d=1 vs t_d=4
t_d=1 vs t_d=5
t_d=2 vs t_d=3
t_d=2 vs t_d=4
t_d=2 vs t_d=5
t_d=3 vs t_d=4
t_d=3 vs t_d=5
t_d=4 vs t_d=5
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

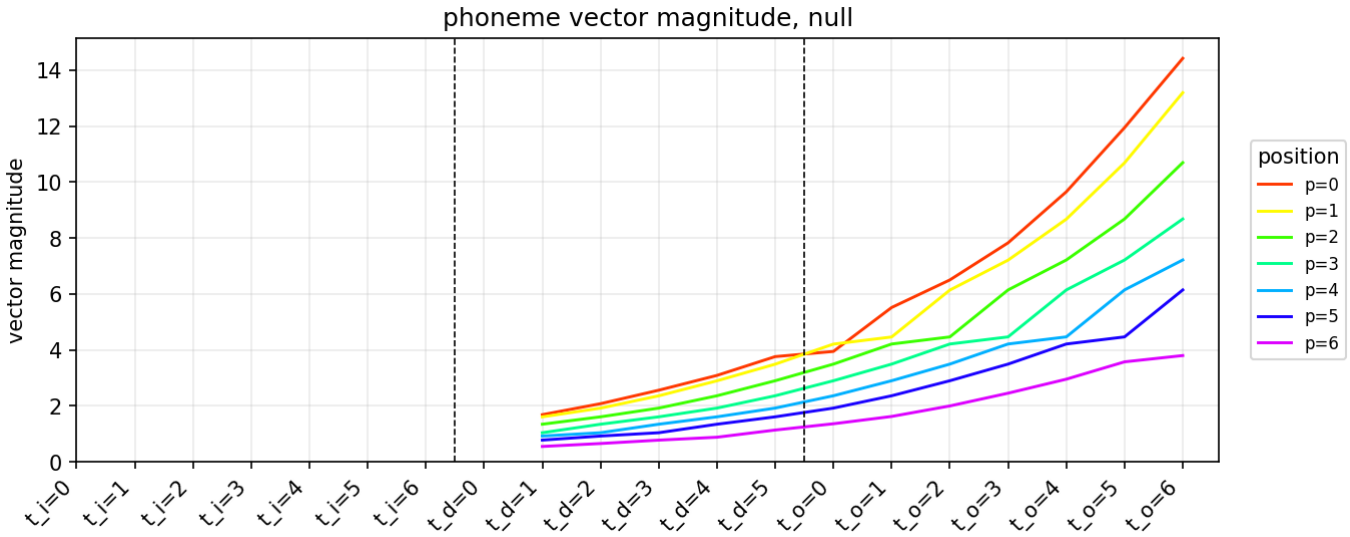
SLIM time-pair position similarity — output — natural cosine
μ=0.1192188959498145, σ=0.2749051641346244, cutoff=none




delay hidden-state transition eigenvalues
$t_d = 0 \to 1,\ \alpha = 0.01$
$t_d = 1 \to 2,\ \alpha = 0.01$
$t_d = 2 \to 3,\ \alpha = 0.01$
$t_d = 3 \to 4,\ \alpha = 0.01$
$t_d = 4 \to 5,\ \alpha = 0.01$
imaginary
real

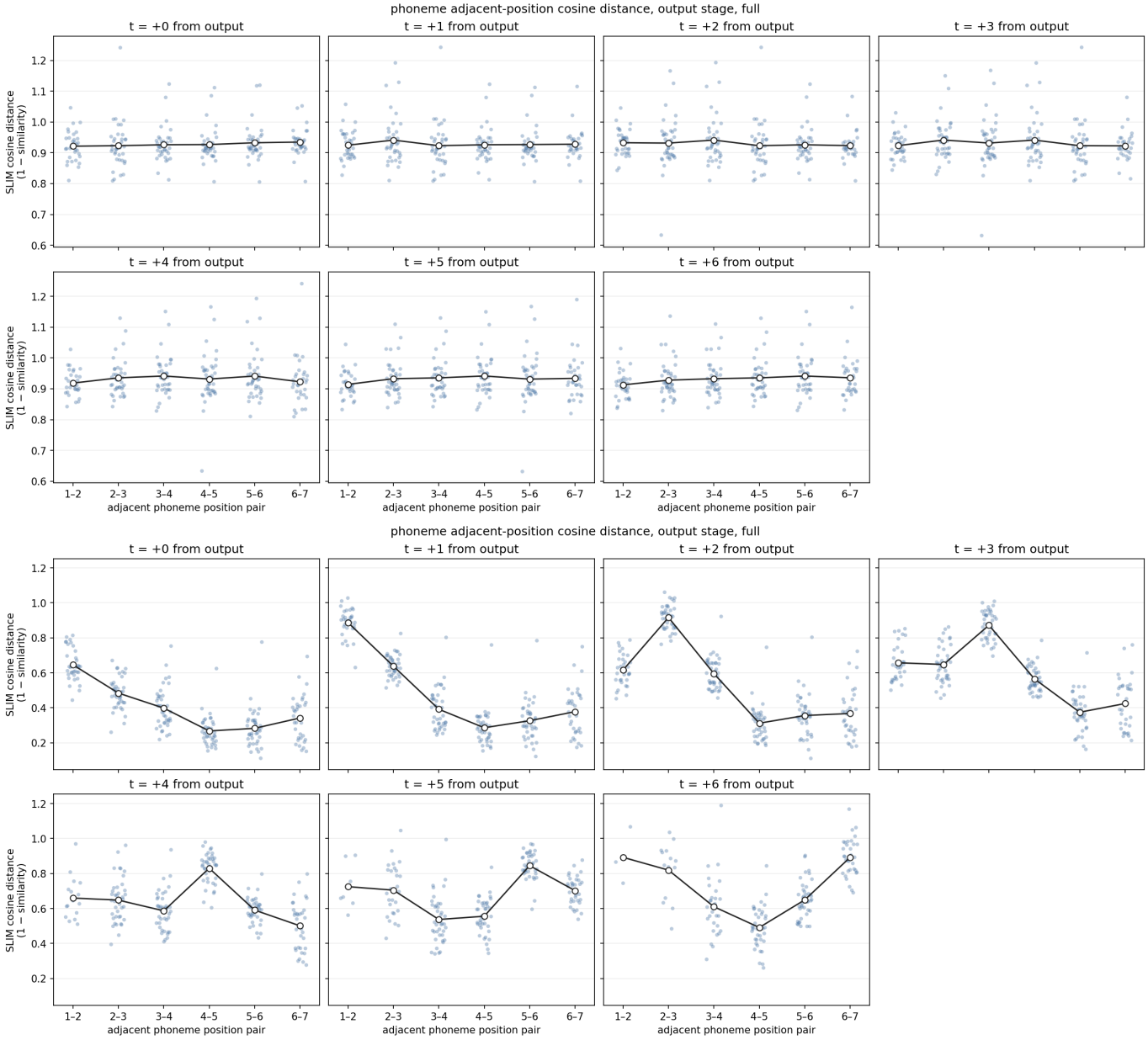

output hidden-state transition eigenvalues
$t_o = 0 \to 1,\ \alpha = 0.01$
$t_o = 1 \to 2,\ \alpha = 0.01$
$t_o = 2 \to 3,\ \alpha = 0.01$
$t_o = 3 \to 4,\ \alpha = 0.01$
$t_o = 4 \to 5,\ \alpha = 0.01$
$t_o = 5 \to 6,\ \alpha = 0.01$
imaginary
real


phoneme vector magnitude, potent
vector magnitude
position
p=0
p=1
p=2
p=3
p=4
p=5
p=6
t_i=0
t_i=1
t_i=2
t_i=3
t_i=4
t_i=5
t_i=6
t_d=0
t_d=1
t_d=2
t_d=3
t_d=4
t_d=5
t_o=0
t_o=1
t_o=2
t_o=3
t_o=4
t_o=5
t_o=6



(Top) Eigenvalues for linear maps fit between time steps during the delay and output period for some $l = 7, m = 6$ linear RNN. Dashed lines represent unit circle; It appears that some dimensions are contracted, others rotated and contracted, and others rotated and expanded. (Bottom) Unlike typical nonlinear solutions, the output-null space representations continue growing in magnitude even after expression.

The linear RNN solution is also characterized by largely orthogonal subspaces, as indicated by the high dissimilarity of neighboring slots. Compare an $l = 7, m = 6, d = 384$ linear model with its nonlinear equivalent; the cosine distance for neighboring phoneme pairs is near 1 (90°) for the nonlinear model, while for the linear model it ranges from approxmately 0.2 to 1.



From this, we can conclude that purely linear computations are unideal for handling the input and storage of diverse sequence lengths. However, it is uncertain whether nonlinear solutions need be disorganized. As discussed next, nonlinear solutions naturally develop organized rotational trajectories with sufficiently high weight decay. Thus, we cannot conclude that nonlinearity $\implies$ partial rotations.

Weight Decay

Finally, we examine the effects of high weight decay. For $d = 384, l \leq 7, m = 0, 2, 4, 6$, and 3 random seeds, we test weight decay values of $10^{-1}, 1, 10, 100$ (previously, we did not observe significant effects with weight decays of 0 to 10^{-2}). We notice that rotation depth increases strongly from weight decays 10^{-1} to 1, while the word error rates remained acceptable at < 0.001. A higher weight decay of 10 and higher resulted in difficulty learning.

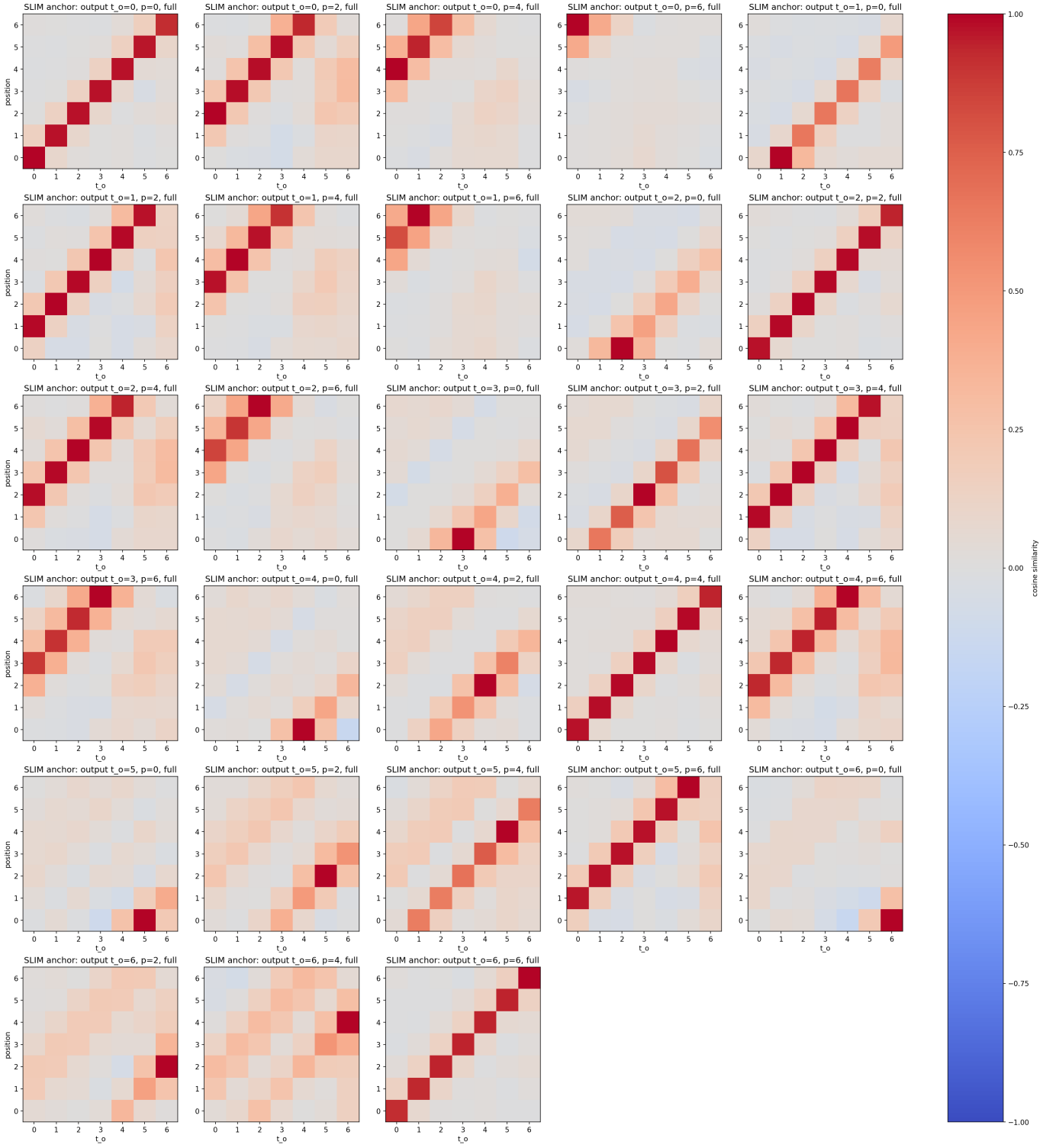
Delay	WD 1e-4	WD 0.1	WD 1	WD 10	WD 100
0	4,4,4 (4.00)	4,5,5 (4.67)	6,5,5 (5.33)	6,6,6 (6.00)	0,0,0 (0.00)

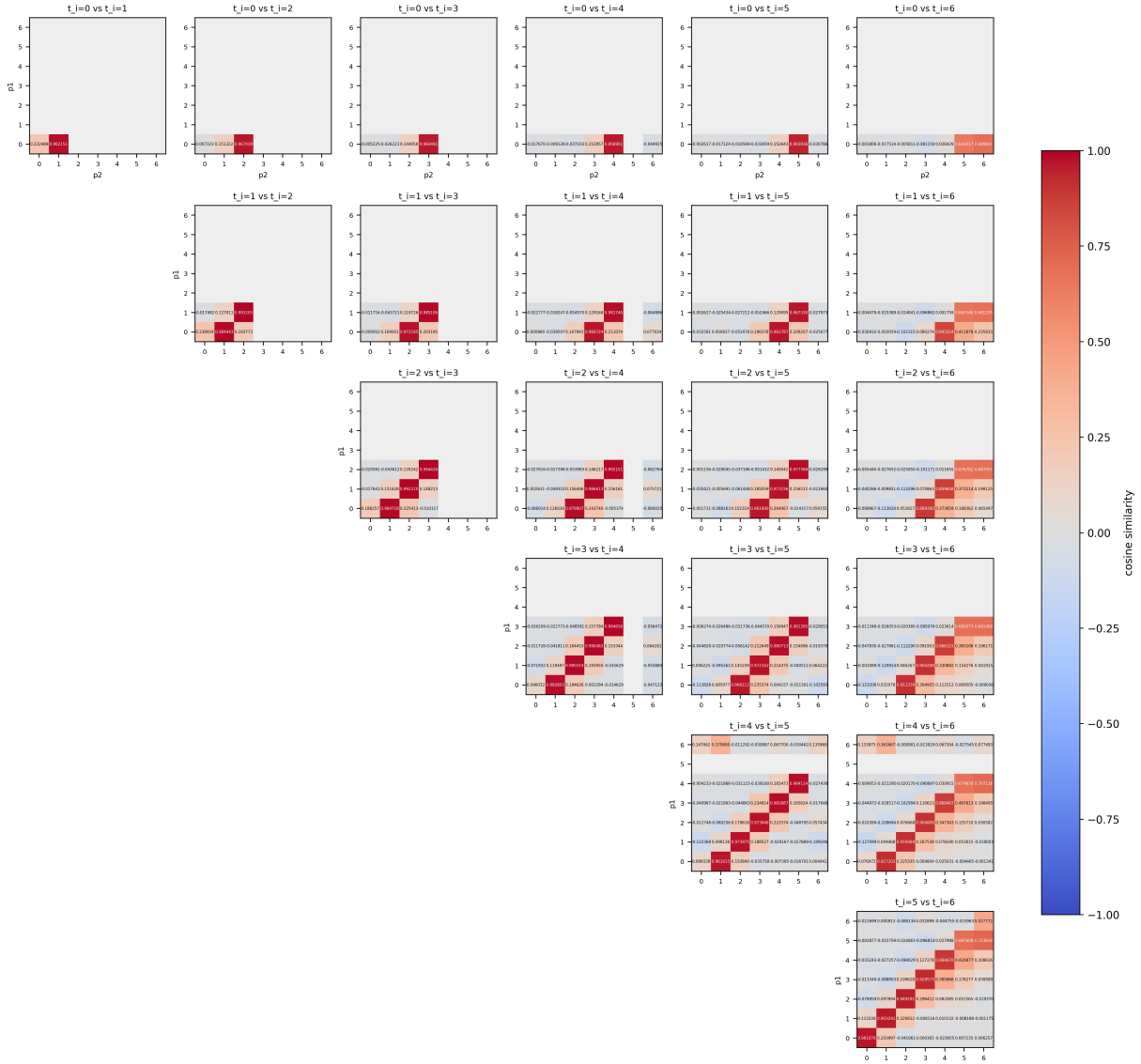
Delay	WD 1e-4	WD 0.1	WD 1	WD 10	WD 100
2	4,4,4 (4.00)	4,4,4 (4.00)	6,6,6 (6.00)	6,6,6 (6.00)	0,0,0 (0.00)
4	3,4,5 (4.00)	4,6,6 (5.33)	6,5,6 (5.67)	0,6,6 (4.00)	0,0,0 (0.00)
6	3,3,4 (3.33)	4,4,3 (3.67)	6,6,6 (6.00)	0,6,0 (2.00)	0,5,1 (2.00)

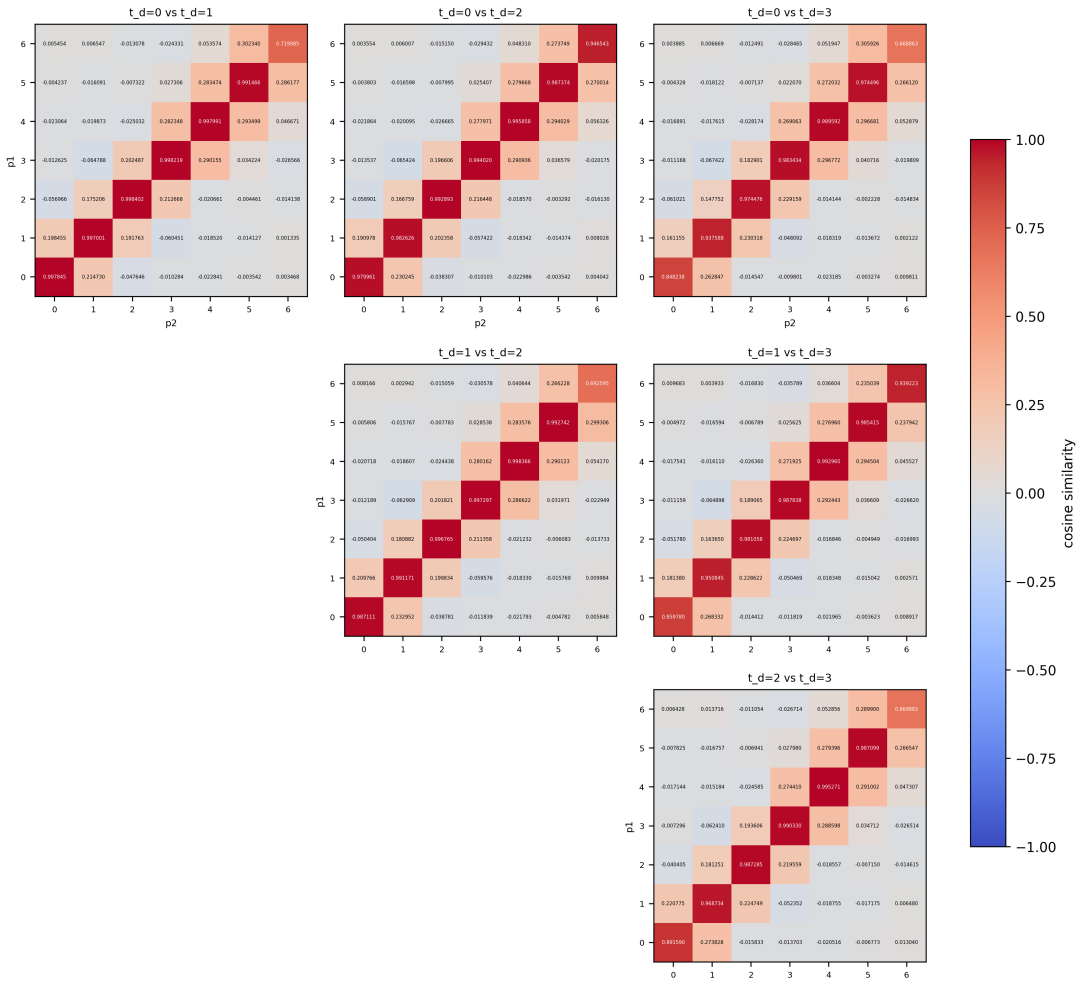
3-seed rotation depths and average, for different weight decay and delay period lengths. We also include a previously tested weight decay of 10^{-4}.

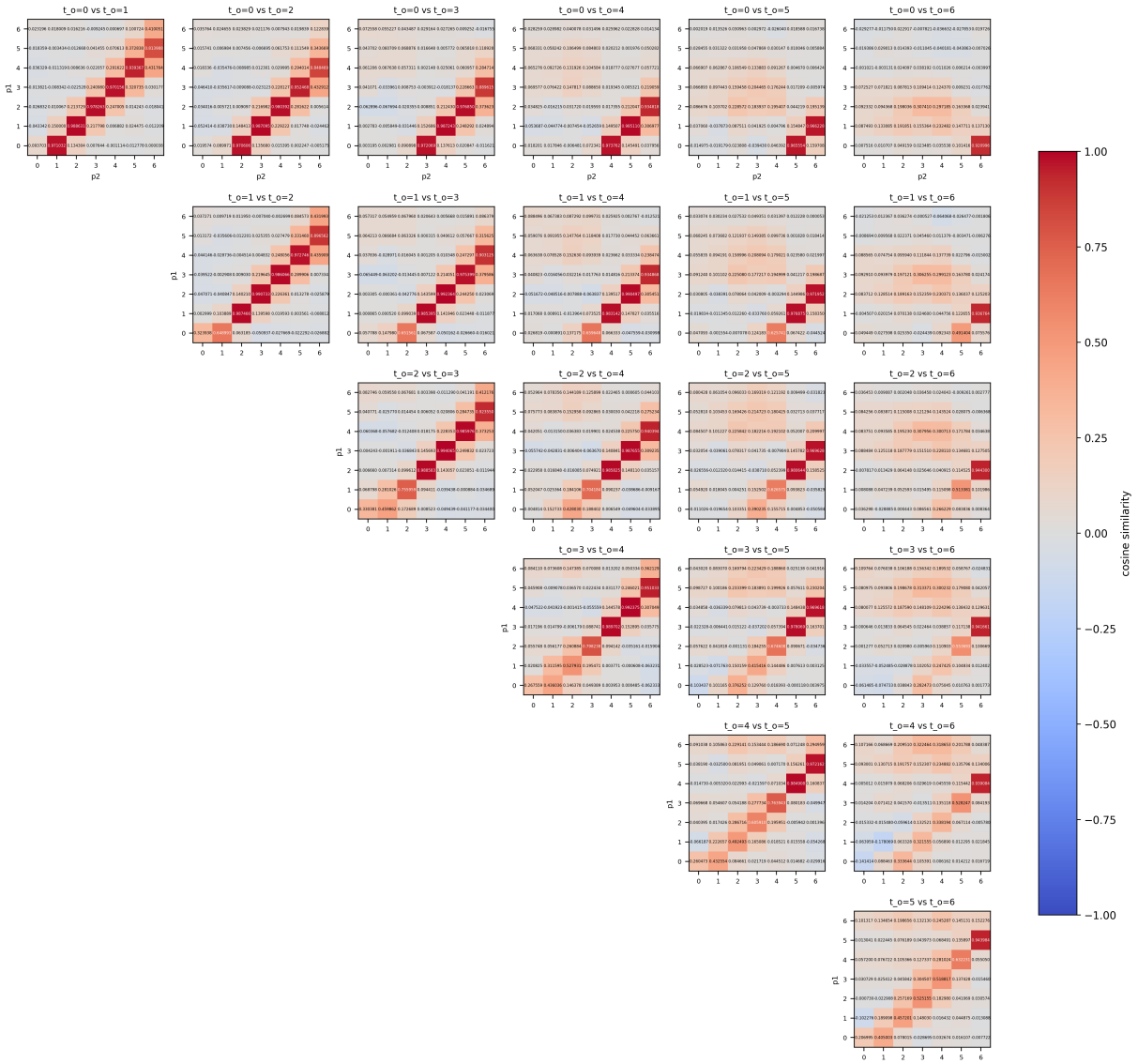
Now, we ask what the weight decay 1 solution looks like: Does it at all resemble the linear solution, or is it just a more organized version of its less regularized equivalents? Unlike the exact length linear models, we observe that, during the delay period, phoneme representations remain fixed, with the representation for each position most similar to that of the same position at other time steps. This is expected, as the GRU architecture can develop separate solutions for hidden state updates during input, delay, and output, and holding the sequences in constant slots is necessary to accommodate for sequences of different lengths. Furthermore, the output stage transition map eigenvalues largely lie within the unit circle. Compared to the linear solutions, we also see a greater similarity and overlap in the representations of adjacent phonemes. However, when compared to less regularized models (weight decay 10^{-1}), the representations are significantly more distinct with less overlap between adjacent phonemes, and each phoneme more compressed in effective dimensionality.

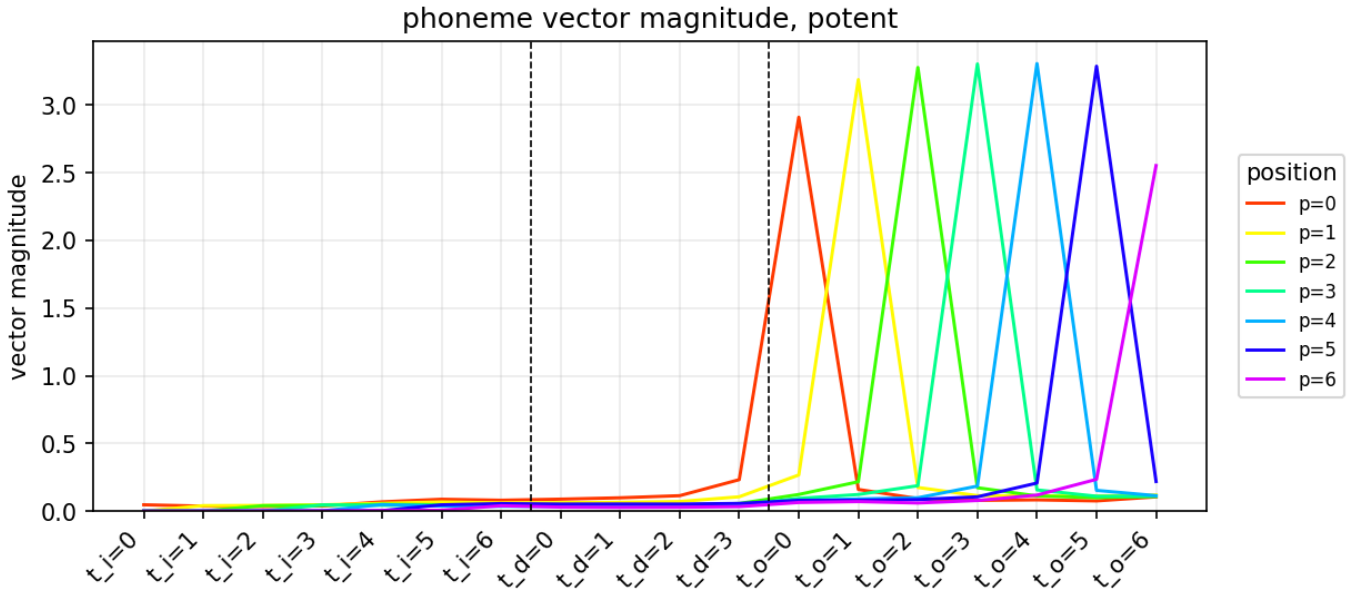
$l \leq 7, m = 4$ weight decay 1 hidden state geometries. (Top) output period representation similarities compared to anchor values. (Center 3) similarities between positions between time step pairs, input, delay, and output period. (Bottom 3) phoneme representation magnitudes in the output-null and output-potent spaces and the output-stage eigenvalue decomposition for linear maps fit to transform hidden states betwen adjacent time steps.

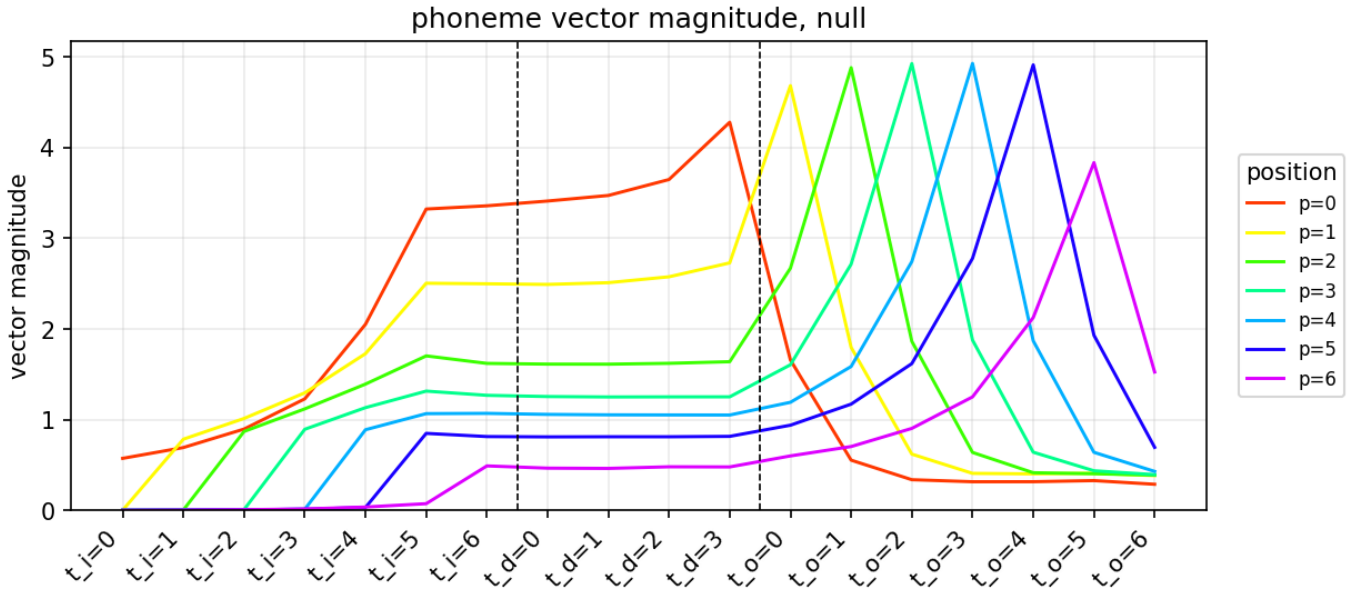

SLIM anchor: output t_o=0, p=0, full
SLIM anchor: output t_o=0, p=2, full
SLIM anchor: output t_o=0, p=4, full
SLIM anchor: output t_o=0, p=6, full
SLIM anchor: output t_o=1, p=0, full
SLIM anchor: output t_o=1, p=2, full
SLIM anchor: output t_o=1, p=4, full
SLIM anchor: output t_o=1, p=6, full
SLIM anchor: output t_o=2, p=0, full
SLIM anchor: output t_o=2, p=2, full
SLIM anchor: output t_o=2, p=4, full
SLIM anchor: output t_o=2, p=6, full
SLIM anchor: output t_o=3, p=0, full
SLIM anchor: output t_o=3, p=2, full
SLIM anchor: output t_o=3, p=4, full
SLIM anchor: output t_o=3, p=6, full
SLIM anchor: output t_o=4, p=0, full
SLIM anchor: output t_o=4, p=2, full
SLIM anchor: output t_o=4, p=4, full
SLIM anchor: output t_o=4, p=6, full
SLIM anchor: output t_o=5, p=0, full
SLIM anchor: output t_o=5, p=2, full
SLIM anchor: output t_o=5, p=4, full
SLIM anchor: output t_o=5, p=6, full
SLIM anchor: output t_o=6, p=0, full
SLIM anchor: output t_o=6, p=2, full
SLIM anchor: output t_o=6, p=4, full
SLIM anchor: output t_o=6, p=6, full
position
t_o
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

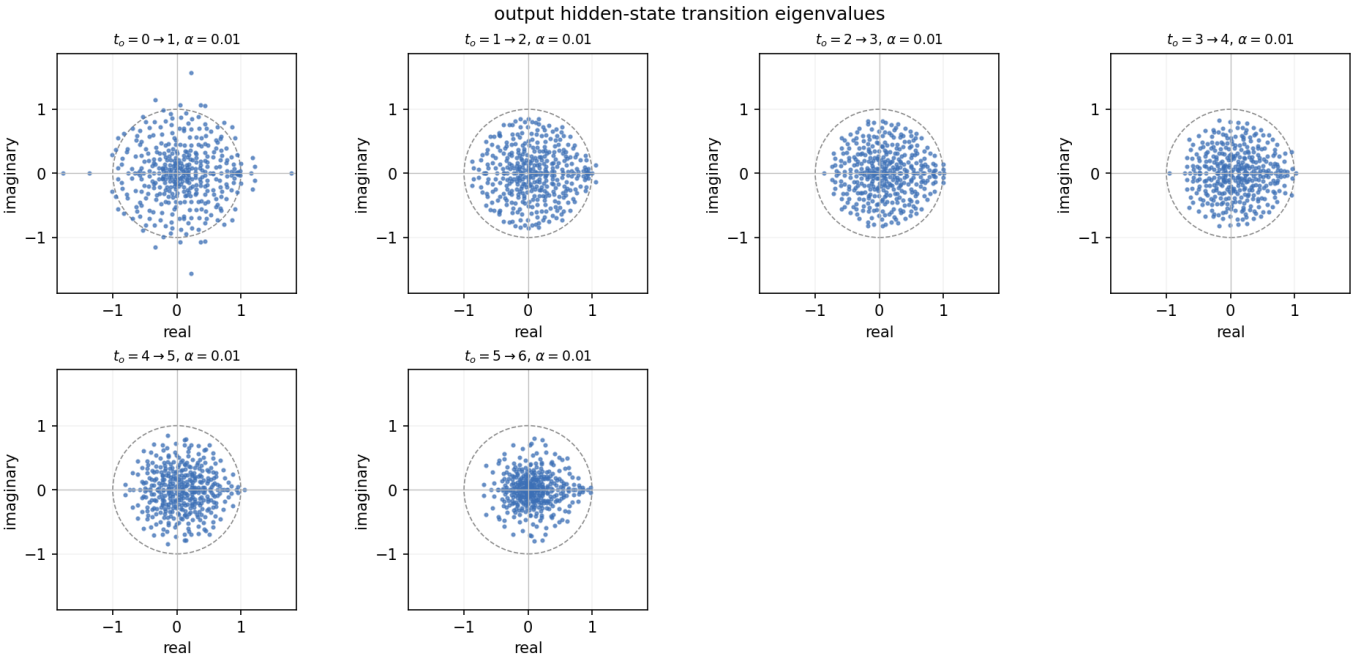

SLIM time-pair position similarity — input — natural cosine
μ=0.18975212387102325, σ=0.34143868316832265, cutoff=none
t_i=0 vs t_i=1
t_i=0 vs t_i=2
t_i=0 vs t_i=3
t_i=0 vs t_i=4
t_i=0 vs t_i=5
t_i=0 vs t_i=6
t_i=1 vs t_i=2
t_i=1 vs t_i=3
t_i=1 vs t_i=4
t_i=1 vs t_i=5
t_i=1 vs t_i=6
t_i=2 vs t_i=3
t_i=2 vs t_i=4
t_i=2 vs t_i=5
t_i=2 vs t_i=6
t_i=3 vs t_i=4
t_i=3 vs t_i=5
t_i=3 vs t_i=6
t_i=4 vs t_i=5
t_i=4 vs t_i=6
t_i=5 vs t_i=6
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00


SLIM time-pair position similarity — delay — natural cosine
μ=0.18904976939402013, σ=0.33051996498670272, cutoff=none
t_d=0 vs t_d=1
t_d=0 vs t_d=2
t_d=0 vs t_d=3
t_d=1 vs t_d=2
t_d=1 vs t_d=3
t_d=2 vs t_d=3
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

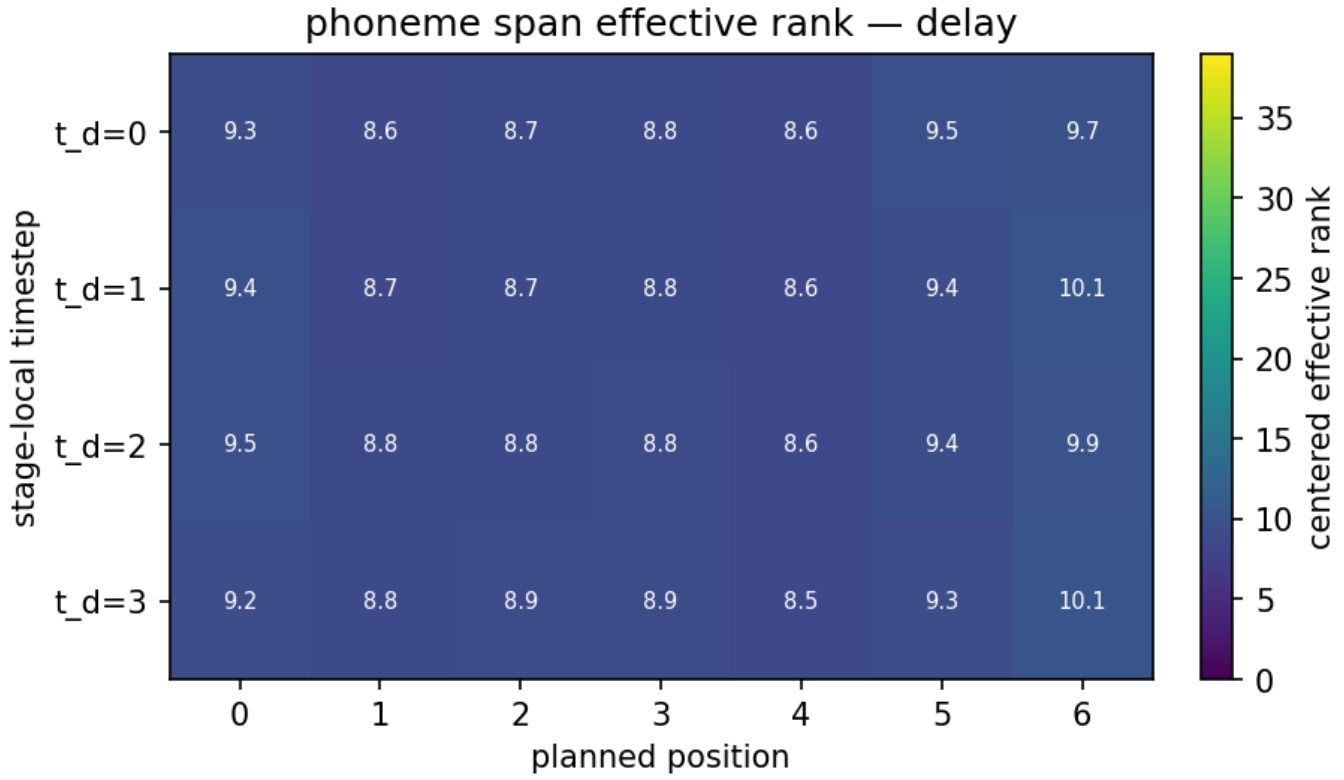

SLIM time-pair position similarity — output — natural cosine
μ=0.13118073393016111, σ=0.23454143740841926, cutoff=none
t_o=0 vs t_o=1
t_o=0 vs t_o=2
t_o=0 vs t_o=3
t_o=0 vs t_o=4
t_o=0 vs t_o=5
t_o=0 vs t_o=6
t_o=1 vs t_o=2
t_o=1 vs t_o=3
t_o=1 vs t_o=4
t_o=1 vs t_o=5
t_o=1 vs t_o=6
t_o=2 vs t_o=3
t_o=2 vs t_o=4
t_o=2 vs t_o=5
t_o=2 vs t_o=6
t_o=3 vs t_o=4
t_o=3 vs t_o=5
t_o=3 vs t_o=6
t_o=4 vs t_o=5
t_o=4 vs t_o=6
t_o=5 vs t_o=6
p1
p2
cosine similarity
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

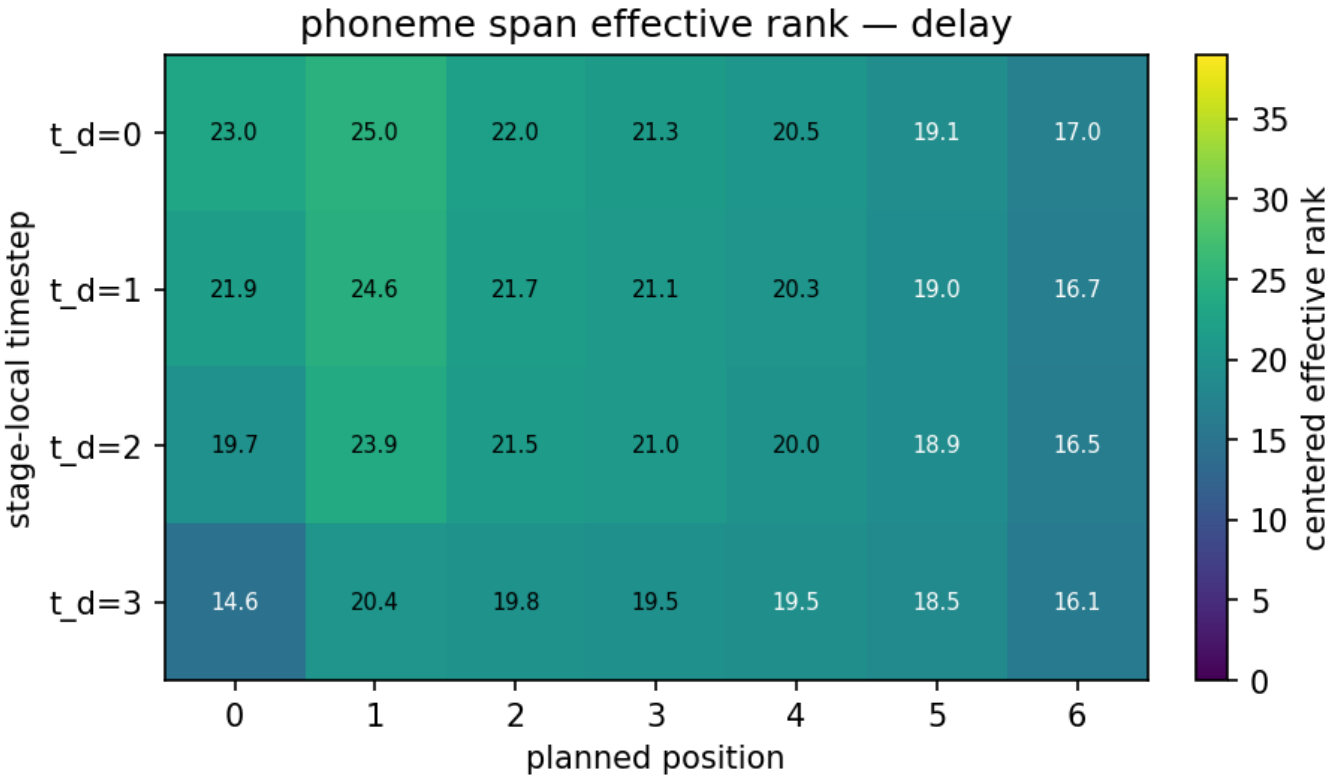

phoneme vector magnitude, potent
vector magnitude
position
p=0
p=1
p=2
p=3
p=4
p=5
p=6
t_i=0
t_i=1
t_i=2
t_i=3
t_i=4
t_i=5
t_i=6
t_d=0
t_d=1
t_d=2
t_d=3
t_o=0
t_o=1
t_o=2
t_o=3
t_o=4
t_o=5
t_o=6

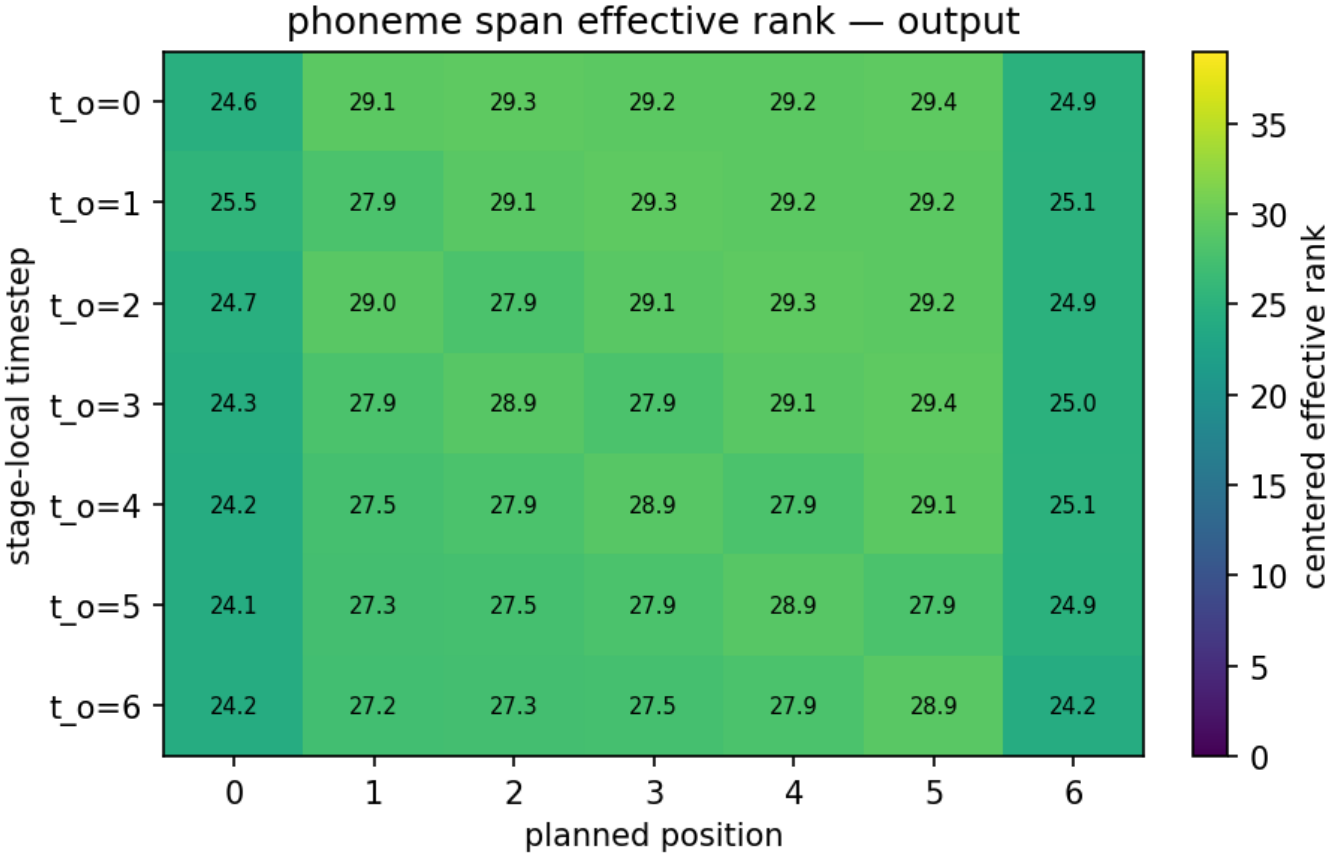

phoneme vector magnitude, null
vector magnitude
position
p=0
p=1
p=2
p=3
p=4
p=5
p=6
t_i=0
t_i=1
t_i=2
t_i=3
t_i=4
t_i=5
t_i=6
t_d=0
t_d=1
t_d=2
t_d=3
t_o=0
t_o=1
t_o=2
t_o=3
t_o=4
t_o=5
t_o=6


output hidden-state transition eigenvalues
$t_o = 0 \to 1,\ \alpha = 0.01$
$t_o = 1 \to 2,\ \alpha = 0.01$
$t_o = 2 \to 3,\ \alpha = 0.01$
$t_o = 3 \to 4,\ \alpha = 0.01$
$t_o = 4 \to 5,\ \alpha = 0.01$
$t_o = 5 \to 6,\ \alpha = 0.01$
imaginary
real

Effective rank of individual phoneme representations during the delay period, weight decay 1 (top), weight decay 10^{-1} (center), and an $l = 7$ linear model (bottom). As the nonlinear weight decay models were trained on $l \leq 7$ sequences, the effective ranks of representations aren't directly comparable. However, it may be notable that higher weight decay forces less overlapping phoneme representations through compressing the representations to lower dimensional spaces, while the linear models separate phoneme representations to near-orthogonal subspaces without this compression.




phoneme span effective rank — delay
stage-local timestep
t_d=0
t_d=1
t_d=2
t_d=3
23.0 25.0 22.0 21.3 20.5 19.1 17.0
21.9 24.6 21.7 21.1 20.3 19.0 16.7
19.7 23.9 21.5 21.0 20.0 18.9 16.5
14.6 20.4 19.8 19.5 19.5 18.5 16.1
0 1 2 3 4 5 6
planned position
centered effective rank
0 5 10 15 20 25 30 35


phoneme span effective rank — output
stage-local timestep
t_o=0
t_o=1
t_o=2
t_o=3
t_o=4
t_o=5
t_o=6
24.6 29.1 29.3 29.2 29.2 29.4 24.9
25.5 27.9 29.1 29.3 29.2 29.2 25.1
24.7 29.0 27.9 29.1 29.3 29.2 24.9
24.3 27.9 28.9 27.9 29.1 29.4 25.0
24.2 27.5 27.9 28.9 27.9 29.1 25.1
24.1 27.3 27.5 27.9 28.9 27.9 24.9
24.2 27.2 27.3 27.5 27.9 28.9 24.2
0 1 2 3 4 5 6
planned position
centered effective rank
0 5 10 15 20 25 30 35